



山东大学
SHANDONG UNIVERSITY

毕业论文（设计）

论文（设计）题目： 基于 HNSW 的高维向量搜索系统

姓 名	王宁波
学 号	202100130176
学 院	计算机科学与技术学院
专 业	计算机科学与技术
年 级	2025 级
指导教师	肖梦白

2025 年 4 月 29 日

摘 要

这里是中文摘要。

关键词: 关键词 1; 关键词 2; 关键词 3; 关键词 4; 关键词 5

ABSTRACT

Here is the English abstract.

Key Words: Key Words 1, Key Words 2, Key Words 3, Key Words 4, Key Words 5

目 录

第一章 绪论

1.1 选题背景和研究意义

随着互联网的发展，在同一时间内有海量可用信息资源的产生，这导致了对高效的信息搜索算法的高需求。传统的信息检索方法具有语义理解能力弱、无法处理非结构化数据、灵活性与扩展性不足等缺点，向量搜索算法应运而生。向量搜索的第一步是各种复杂的数据转换为具有不同维度的向量，向量的维数代表数据的不同特征，维度越多，能捕捉到的信息就越多，区分也就越细致，但同时也会增加计算需求。向量搜索算法通过计算不同向量的距离（余弦相似度、欧式距离等）来判断数据的相似度，与传统的依赖关键字匹配和统计技术不同，向量搜索可以理解潜在的含义或上下文，并且可用于包括文本、图像、音频、结构化数据、多模态数据等多种数据。基础的向量搜索算法为最近邻搜索，通过计算查询与数据集中每个元素之间的距离选择距离最小的元素，然而这种简单方法的复杂度随着存储元素数量线性增长，使其在大规模数据集上不可行，并且随着向量的维度增加，精确搜索的计算资源消耗呈指数级增长，因此发展出了近似最近邻搜索算法。近似最近邻搜索通过允许少量误差来放宽精确搜索的条件，进而大大减少向量查询时间、提高查询效率。

1.2 研究现状

近似最近邻算法可以分为：基于树结构的算法，通过空间划分快速剪枝搜索范围如 KD-Tree、Annoy；基于哈希的算法，如局部敏感哈希（LSH）利用哈希函数将相似数据映射到相同桶中实现快速粗筛；基于量化的算法（如 PQ、SQ），通过压缩高维向量降低存储和计算开销；基于图索引的算法，通过邻居关系建立图索引结构，并在建立的索引的基础上进行查询，通过贪心策略逼近最近邻，如多层可导航小世界图（HNSW）算法、倒排索引算法等，相较于暴力搜索，索引结构能快速过滤掉距离较远的数据点从而提高搜索效率，本论文的主要任务就是基于 HNSW 索引设计并实现一个高效的高维向量搜索系统。

HNSW 算法最早由 Y. A. Malkov 等人在“Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”中提出，HNSW 作

为一种基于图的向量索引，其核心思想是在可导航小世界（NSW）的基础上进一步引入基于跳表的多层次结构，每一层包含若干数据点，自高到低每一层的数据点数逐渐增多，最底层包含全部节点，每个数据点都与一系列相邻节点连接，形成一个多层次的图。在 HNSW 中，每一层都是一个 NSW 图，通过这种分层图结构，搜索过程可以从较高层次的“粗略”搜索开始，然后逐步深入到较低层次的“精确”搜索，通过这种逐步精细化的图层次搜索，HNSW 保证了查询效率和结果的准确性，最终返回查询点的 k 个近似最近邻。论文中作者将 HNSW 与多种搜索算法进行了对比，结果表明 HNSW 在搜索速度上明显优于包括 NSW 算法在内的大部分算法，并且与基于乘积量化的算法的对比结果表明尽管 HNSW 需要更多的内存，但它可以实现更高的准确性，同时在搜索速度和索引构建速度方面有了巨大的进步。本文就是在该文献的基础上，设计并复现了 HNSW 的数据结构和搜索算法并实现在大规模向量数据中的高效并发搜索。

1.3 论文组织结构

本文共分为五章，其中每章具体的内容如下：第一章为绪论，讲述了本课题的提出背景与研究意义，并指出目前本课题的研究现状，指明本文的结构。第二章讲述了与本论文相关的理论与研究，主要有 Delaunay 图与近邻图技术、可导航小世界图、分层可导航小世界图。第三章是 HNSW 树的具体实现。给出了每一步实现的思路以及伪代码，包括数据结构设计、HNSW 索引构建、向量搜索、索引保存和加载、性能优化等方法。第四章是该 HNSW 向量搜索系统的参数敏感性实验以及结果分析。第五章是全文的总结和概括。

第二章 相关工作与算法描述

2.1 Delaunay 图

Delaunay 图是基于 Delaunay 三角剖分构建的。在二维空间中, 给定一组点, Delaunay 三角剖分会把这些点连成一系列三角形, 使得没有任意一个点在任意一个三角形的外接圆内, Delaunay 图的边就是所有这些三角形的边的集合。因为它本质上就是对整个点集的空间划分, 没有割裂, 所以 Delaunay 图可以保证图中任何节点之间都可以通过直接或者间接的路径连接起来, 且能保证整张图的边的数量尽可能的少。基于 Delaunay 图的连通性, 一些基于图的搜索算法会通过近似 Delaunay 图的形式构造 k 近似图实现近似最近邻搜索, 但这些近似 Delaunay 图的方法存在一些缺点, 比如随着数据集增大, 贪心路由所需步数增长很快, 导致大规模数据搜索效率变低, 如果数据分布有明显簇, k 近似图可能出现不连通或分块, 导致搜索被困在某个区域, 找不到全局最优结果。

2.2 可导航小世界

小世界现象, 最早由社会学家米尔格拉姆在 1960 年代提出。它描述的是: 在一个大型网络中, 任意两个人之间通过很少的“中间人”就能建立联系, 平均路径很短。“小世界网络”具备两个典型特征: 某个节点的邻居之间, 往往彼此也是邻居, 形成很多“小圈子”或三角关系, 并且任意两个节点之间的最短路径很短, 通常随着网络规模增加, 路径长度只以对数方式缓慢增加, 而不是线性或更快地增长。

基于小世界网络的思想, 提出了 NSW (可导航小世界) 算法用于近似最近邻搜索, 在构建时, 将节点按照随机顺序插入, 每插入一个新节点时, 从若干随机选取的入口点开始, 在当前已有的图中贪心搜索, 找到与新节点距离最近的 M 个节点, 将新节点与这 M 个最近邻节点双向相连。在刚开始构图的时候, 对于一个新插入的点, 在寻找哪些图中已有的点跟它建立连接的时候, 它的候选很少, 因为图才刚开始构建, 有可能它与已有的点距离其实很远, 但是它们还是建立了连接, 在后期图中的节点已经很多了, 那么新插入一个点的时候, 它更有可能找到离它真的很近的 M 个节点。

在构建 NSW 图结构的时候, 通过寻找 M 个最近邻来建立类似于 Delaunay 三角剖分的结构, 由于插入顺序的随机性, 在早期建立的随机长边可以将两个距离较远的点直

接连接起来，而不需要经过多个节点的搜索转发，能够实现在不同区域之间快速跳转，后期建立的短边可以保证当我们检索到一个近邻节点时，其大部分近邻节点都是近邻节点，这样就构建了一个可导航的小世界网络。

NSW 算法的搜索分为两个阶段，我们首先通过长连接进行放大阶段，在该阶段我们通过低度顶点进行传递，然后进行缩小阶段，在该阶段我们通过高度顶点进行传递，停止条件是在当前顶点的朋友列表中找不到更近的顶点。因此在放大阶段因为节点具有较少的链接，更不可能找到更近的顶点，我们更有可能遇到局部最小值并过早停止，可以增加顶点的平均度数的方式最小化过早停止的概率，并增加召回率。

NSW 的距离计算次数等于搜索要经过的节点数量乘以每个节点的度，这两项随着图规模的增大，都是对数增长的，两个对数复杂度相乘，就是多重对数复杂度。对于具有聚类效应的点，由于后续插入的点可能都和其建立连接，对应节点的度可能会比较高，为了避免遇到局部最小值并过早停止，还需要增加顶点的平均度数，因此在一个大规模的小世界网络中进行检索，尽管有长连接机制的存在耗时还是很高。

2.3 分层可导航小世界

分层可导航小世界 (HNSW) 的实现参考了跳表的原理，跳表的作用可以理解为，先快速的接近要查找的点的附近，然后再精确的搜索，这样就避免了路上做很多无用功耽误时间。分层可导航小世界将跳表运用在了图结构中，跳表的每一层，都是一个小世界网络。其中最底层是一个完整的可导航小世界图，包含了连通图中所有的点。HNSW 通过将把节点分散到不同层级保证检索要经过的节点数量依旧对数增长，而节点的度是常量，那么整个检索过程就是对数增长而不是多重对数增长了。

形成这种分层结构的方式是在将元素插入之前随机选取元素的最大层级，插入时先从顶层图的入口点开始贪婪查找节点的最近邻，并将其作为下一层遍历的入口点，直到进入节点最大层级，此时贪婪地查找最近的一些节点，并从中选择邻居建立连接，重复这一过程直至最底层，其中选择邻居用到了启发式裁剪的方法。

HNSW 的搜索过程从顶层开始，在顶层我们可以找到最长的链接，而在底层有最短的链接，这意味着从 NSW 所描述的放大阶段开始，并且因为跨越多层，这些顶点往往是高度顶点。在查找时在每一层进行贪婪搜索，直至达到局部最小值（最近邻），此时并不会提前停止而是将其作为下一层的入口点进入下一层重复搜索，直至到达最底层，在最底层查找并返回 K 个最近邻

第三章 hns w 实现描述

3.1 环境配置

编程语言：go1.24.1。处理器：AMD Ryzen 7 5800H with Radeon Graphics 3.20 GHz
系统：Windows 11 专业版 24H2

3.2 数据结构设计

3.2.1 node 结构体

结构体 node 代表 HNSW 图中的一个节点，vector []float32 是一个用于存储浮点数类型切片，类似于其他语言中的数组，在这里存储的是向量本身；connections [][]uint32 是一个二维切片，用于表示节点的邻居关系，外层切片的每个元素对应一个层级，每个子切片存储了当前节点在某一层级上的邻居节点的 ID；level int 表示插入节点时分配的层级；sync.RWMutex 用于保护共享数据的并发访问

算法 3-1: 结构体 node

1 sync.RWMutex	// 读写锁
2 vector []float32	// 点
3 level int	// 最大层级
4 connections [][]uint32	// 朋友列表

3.2.2 Hnsw 结构体

一个 HNSW 包含一个 node 类型的切片用于存储所有节点的索引，此外还包含一个入口点，搜索通常从入口点开始，然后逐层向下进行；位集合池 bitset 用于在搜索过程中标记已访问的节点，DistFunc 用于计算节点间的距离；此外还包含一些构建索引时的图的参数配置，其中 M 控制每个节点在图中最多可以连接的邻居数量，较大的 M 值通常会提高搜索精度，但会增加内存使用和构建时间，M0 通常是 M 的一个较大值，用于控制图中底层的邻居数量，efConstruction 是构建图时的搜索宽度，它决定了在构建过程中搜索的候选邻居数量，较大的 efConstruction 值会提高图的质量，但也会增

加构建时间，LevelMult (mL) 用于代入公式 $-\ln(\text{unif}(0..1)) \text{ mL}$ 中为新节点分配层级

算法 3-2: 结构体 node

```

1 sync.RWMutex                                // 读写锁
2 M int                                         // 参数 M
3 M0 int                                       // 参数 M0
4 efConstruction int                           // 构建时的 ef 参数
5 DelaunayType int                            // 控制是否采用启发式裁剪选择邻居
6 DistFunc func([]float32, []float32) float32 // 距离函数，返回两个节点之间的
    欧几里得距离
7 nodes []node                                 // 节点列表
8 bitset *bitsetpool.BitsetPool               // 位集合池
9 LevelMult float64                           // 层级乘数，用于分配层级
10 maxLayer int                                // 最大层级
11 enterpoint uint32                           // 入口点，初始化 id 为 0

```

3.3 HNSW 索引构建

3.3.1 searchAtLayer 层搜索

searchAtLayer 用于搜索节点在某一层的最近邻，它的做法是：先把入口点加入已访问列表和候选队列（最小堆，按距离排序），然后不断从候选队列里挑出当前最近的节点，先计算它与目标节点的距离，循环结束条件是如果当前节点的距离比结果集中最远距离 lowerBound 还大，跳出循环，因为即使遍历它们的邻居，也不可能再找到比 lowerBound 更近的点，因为所有未访问的点都只能通过这些更远的节点扩展出来，通过这些更远节点的邻居，也不可能找到比当前结果集更近的点（因为所有更近的点要么已经被访问过，要么已经在结果集中）。如果比 lowerBound 小，看它在这一层的邻居有没有没访问过的。如果有，就算一下距离，如果更近或者结果集还没满，就把它也加入候选队列和结果集。这样一轮一轮扩展，直到没有更近的节点为止。整个过程其实就是在这一层做一个受限的最短路搜索，尽量找到和目标向量最接近的节点集合，其中已访问列表会用到位集合池的方法。

算法 3-3: searchAtLayer 方法伪代码

Input: 查询向量 q , 结果集 $resultSet$, 扩展因子 $efConstruction$, 入口点 ep , 层级 $level$

Output: 更新后的 $resultSet$

```

1  初始化位集合池  $pool$ 、访问标记  $visited$  和候选集  $candidates$ 
2  将  $ep.ID$  标记为已访问, 加入  $candidates$  和  $resultSet$ 
3  while  $candidates$  不为空 do
4      获取  $resultSet$  中最差距离  $lowerBound$ 
5      弹出  $candidates$  中最近的节点  $c$ 
6      if  $c.D > lowerBound$  then
7          break // 如果当前节点的距离大于最差距离, 跳出循环
8      for  $c$  的朋友列表中的每个邻居  $n$  do
9          if  $n$  未访问 then
10             标记  $n$  为已访问, 计算距离  $d$ 
11             if  $resultSet$  长度  $< efConstruction$  then
12                 将  $n$  加入  $resultSet$  和  $candidates$ 
13             else
14                 if  $d < resultSet$  中最差距离 then
15                     替换最差节点为  $n$ , 加入  $candidates$ 
16 释放  $pool$ 
17 return  $resultSet$ 
    
```

3.3.2 生成随机层级

$$level = \lfloor -\log(rand()) \cdot LevelMult \rfloor \quad (3.1)$$

其中 $LevelMult$ 为传入的参数, 一般设置为 $\frac{1}{\ln M}$, $-\log(rand())$ 服从指数分布, 指数分布是一种“越大越罕见”的分布, 绝大多数点只会被分配到底层, 只有极少数点会被分配到高层, 这样可以保证高层稀疏、底层稠密, 形成“金字塔”结构, 有利于加速搜索, 使得搜索时可以先在高层快速缩小范围, 再逐层下沉到低层精细查找, 兼顾速度和准确

率。

3.3.3 启发式选择邻居

算法 3-4: getNeighborsByHeuristic 伪代码

Input: 候选集 $resultSet$, 最大数量 M

Output: 筛选后的 $resultSet$

```

1 if  $|resultSet| \leq M$  then
2   return
3 初始化临时队列  $tempList$  和结果集  $result$ 
4 while  $resultSet$  非空且  $|result| < M$  do
5   从  $resultSet$  弹出  $e$ ,  $good \leftarrow \text{true}$ //合适邻居标识
6   for  $r$  in  $result$  do
7     if 距离  $(r, e) < d_e$  then
8        $good \leftarrow \text{false}$ ; break//如果  $e$  到  $r$  的距离大于所有结果集中元素到  $r$ 
       的距离
9   if  $good$  then
10     $result \leftarrow result \cup \{e\}$ 
11  else
12     $tempList \leftarrow tempList \cup \{e\}$ 
13 while  $|result| < M$  且  $tempList$  非空 do
14    $result \leftarrow result \cup \{\text{弹出 } tempList\}$ 
15 重置  $resultSet$ 
16 for  $item$  in  $result$  do
17    $resultSet \leftarrow resultSet \cup \{item\}$ 
    
```

启发式选择邻居的工作主要是优先选择那些与已选邻居距离较远的节点，保证其不会被其他更近的元素遮挡，避免结果中出现过于密集的邻居，从而提升图的多样性和搜索效率，通过建立尽量多的“高速公路”，增强图的连通性。对于候选集中的元素，判断该元素距离目标节点的距离是否小于该元素离结果集所有元素的距离，如果成立说明这

个不合适。在启发式选择中，不仅考虑节点之间的距离，还考虑整个图中不同区域的连通性。这样可以更快地对不同区域进行查询，从而减少贪婪搜索的次数。如图 3.1 所示

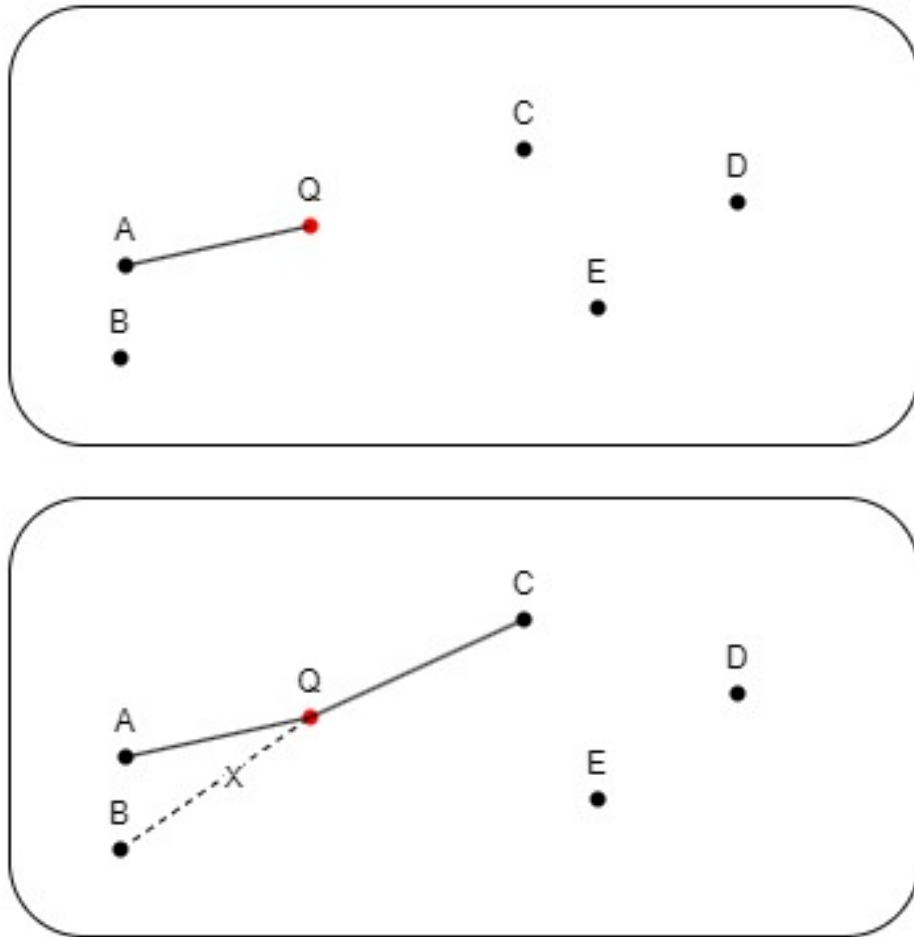


图 3.1 红点 Q 是新插入的点，四个黑点 ABCD 是本次召回的点。首先，我们必定选择 A，因为 A 距离 Q 最近。但第二步，根据最近邻原则，必须选择 B，显然 B 是第二近的。此时形成分界，ABQ 成为一派，CD 成为另一派。启发式剪枝在于：此时 AQ 已确立，下一轮确立时，最近的是 B，但 AB 的距离小于 BQ，因此放弃 B，转而查看次近点 C；CA 的距离大于 CQ，于是 Q 将与 C 建立连接

3.3.4 连接节点

用于在指定层级将两个节点建立连接，并根据最大连接数 (maxL) 和 Delaunay 策略，动态维护节点的邻居列表，算法既保证了图的稀疏性，又能动态维护每个节点的高质量

邻居集合，是 HNSW 算法高效搜索和插入的基础。邻居列表的维护和裁剪是 HNSW 算法性能和准确率的关键环节。

算法 3-5: Link 方法伪代码

Input: 节点 *first*, 节点 *second*, 层级 *level*

Output: 在指定层级将 *second* 链接到 *first*

```

1 设置最大链接数  $maxL \leftarrow M$ , 若  $level = 0$  令  $maxL \leftarrow M0$ 
2 获取节点 first
3 if 未分配足够的朋友列表 then
4   //level 层还未链接邻居扩展到  $level + 1$  层, 每层容量为  $maxL$ 
5   在  $level$  层插入 second
6 else
7   在  $level$  层添加 second
8 if 朋友数量  $> maxL$  then
9   if DelaunayType=0 then
10    构建距离队列 resultSet, 按距离保留最近  $maxL$  个节点
11    更新朋友列表为 resultSet 中最近的  $maxL$  个节点
12  else
13    构建距离队列 resultSet, 调用启发式筛选最近  $maxL$  个节点
14    更新朋友列表为 resultSet 中最近的  $maxL$  个节点

```

3.3.5 添加节点

第一次遍历，自顶向下查找入口点。HNSW 在插入节点时先从最高层开始，逐层贪婪地查找当前层离目标点最近的点，直至到达节点分配层级的上一级，每次都尝试从当前入口点的所有邻居中，寻找距离查询点 *q* 更近的节点。如果找到了，就把入口点 *ep* 更新为这个更近的节点，并把 *changed* 设为 *true*，表示需要继续在新入口点的邻居中查找是否还有更近的节点；当所有邻居都没有比当前入口点更近的节点时，*changed* 就不会被设为 *true*，循环终止。此时，当前 *ep* 就是该层级下距离查询点 *q* 最近的入口点，然后开始下一层的遍历。

第二次遍历，分层邻居搜索与连接。从新节点的层级到第 0 层，依次在每一层调用 *searchAtLayer* 方法，找到该层的 *efconstruction* 个候选邻居。根据 *DelaunayType* 参数，

采用不同的启发式方法筛选出最多 M 个最优邻居，并将这些邻居的 id 存入新节点的连接列表。

最后从新节点的层级到第 0 层，在每一层调用 $Link()$ 方法，新节点与邻居建立双向连接。如果新节点的层级超过当前最大层级，则更新最高层级和入口点分别为新节点的层级和新节点，保证后续搜索和插入的正确性。

算法 3-6: Add 方法伪代码

Input: 向量 q , 编号 id

Output: 将新点加入 HNSW 索引

```

1 if  $id = 0$  then
2   | return // 0 号为保留编号
3 随机生成层级  $curlevel$ 
4 获取入口点  $ep$  及其最大层级  $currentMaxLayer$ 
5 创建新节点  $newNode$ 
6 for  $level = currentMaxLayer$  to  $curlevel + 1$  do
7   | while 入口点  $ep$  可被更近点更新 do
8     | |  $ep \leftarrow$  距离  $q$  最近的邻居
9 for  $level = \min(curlevel, currentMaxLayer)$  to 0 do
10  | 初始化结果集  $resultSet$ 
11  | 在  $level$  层用  $ep$  搜索近邻, 得到  $resultSet$ 
12  | if  $DelaunayType$  为 0 then
13  | | 保留  $M$  个最近元素
14  | else
15  | | 用启发式方法筛选  $M$  个最近元素
16  | 将  $resultSet$  结果作为新节点在  $level$  层的朋友
17 加锁, 将新节点插入到节点集合
18 for  $level = \min(curlevel, currentMaxLayer)$  to 0 do
19  | for 每个朋友  $n$  do
20  | | 建立  $n$  到新节点的连接
21 加锁, 若  $curlevel > maxLayer$ , 则更新  $maxLayer$  和  $enterpoint$ 

```

3.4 向量搜索

3.4.1 search

从最高层逐层往下遍历，从每层的进入点开始，遍历其所有邻居节点，寻找距离查询向量更近的节点。如果找到更近的邻居，则更新入口点，并继续在该层重复这一过程，直到没有更近的邻居为止。这样可以保证每一层都尽可能找到距离查询点最近的入口点，为底层的精细搜索提供更优的起点；当到达第 0 层后，调用 `searchAtLayer` 方法，在第 0 层进行更精细的近邻搜索，将结果存入 `resultSet`，最大不超过 `ef`。最后，如果结果集中的元素数量超过了目标 K ，则弹出最远的元素，只保留最近的 K 个近邻，并返回结果集。

算法 3-7: Search 方法

Input: 查询向量 q ，搜索宽度 ef ，近邻数 K

Output: K 近邻结果集 $resultSet$

```

1 获取最大层级  $currentMaxLayer$ ，入口点  $ep$ 
2 初始化结果集  $resultSet$ 
3 for  $level = currentMaxLayer$  to 1 do
4   while 存在更近点 do
5      $ep \leftarrow$  距离  $q$  更近的邻居
6 在 0 层用  $ep$  搜索，得到  $resultSet$ 
7 while  $|resultSet| > K$  do
8   移除最远的元素
9 return  $resultSet$ 

```

3.4.2 搜索

评估近似搜索 (Search) 与暴力搜索 (SearchBrute) 之间的精度，其核心思想是：用两种方法分别查找查询向量 q 的 K 个最近邻，然后统计近似搜索结果中有多少个节点也出现在暴力搜索的真实结果中，最终返回召回率 (recall)，其中暴力搜索思想是遍历所有节点，计算每个节点与查询向量的距离，并动态维护一个只包含 K 个最近邻的结果集。

算法 3-8: Benchmark 方法伪代码

Input: 查询向量 q , 扩展因子 ef , 近邻数 K

Output: 准确率 acc

```

1 用 Search 方法得到结果集  $result$ 
2 用 SearchBrute 方法得到真实结果集  $groundTruth$ 
3 将  $groundTruth$  转为列表  $truth$ ,  $p \leftarrow 0$                                 // 匹配计数
4 for 每个  $i$  in  $result$  do
5     for  $j = 1$  to  $K$  do
6         if  $truth[j] = i.ID$  then
7              $p \leftarrow p + 1$ 
8  $acc \leftarrow p/K$ 
9 return  $acc$ 

```

3.5 索引保存与加载

3.5.1 索引保存

算法 3-9: Save 方法伪代码

Input: 文件名 $filename$

Output: 将索引数据保存到文件

```

1 创建文件  $filename$ , 并用  $gzip$  包装写入器, 写入当前时间戳
2 依次写入参数
     $M, M0, efConstruction, linkMode, DelaunayType, LevelMult, maxLayer, enterpoint$ 

3 写入节点总数  $l$ 
4 for 每个节点  $n$  do
5     写入向量维度, 写入向量数据, 写入节点层级。写入连接层数
6     for 每层连接  $f$  do
7         写入该层的朋友数量, 写入连接数据
8 关闭写入器和文件
9 return 成功或错误状态

```

3.5.2 索引加载

算法 3-10: Load 方法伪代码

Input: 文件名 *filename*

Output: 恢复的索引 *h*, 时间戳 *timestamp*

1 打开文件 *filename*, 用 gzip 解压并读取时间戳 *timestamp*

2 新建 HNSW 实例 *h*

3 依次读取参数

M, M0, efConstruction, linkMode, DelaunayType, LevelMult, maxLayer, enterpoint

4 初始化距离函数和 bitset 池

5 读取节点总数 *l*, 并分配 *h.nodes*

6 **for** 每个节点 *i* **do**

7 读取向量维度, 分配并读取向量数据, 读取节点层级, 读取连接层数, 分配
连接数组

8 **for** 每层连接 **do**

9 读取该层连接数量, 读取连接数据

10 关闭解压器与文件

11 **return** *h, timestamp*

Save 方法将整个 HNSW 索引结构（包括参数、所有节点的向量、层级、邻居关系等）压缩保存到一个二进制文件中，实现模型的持久化。而 Load 方法则能从这个文件中完整还原出 HNSW 索引和写入时的时间戳，让你可以随时恢复和继续使用之前训练好的索引。两者配合，让大规模向量检索模型的保存与加载变得高效又方便。

3.6 加载 SIFT 数据集

3.6.1 加载数据点

算法 3-11: loadDataFromFvec 伪代码

Input: 文件前缀 *prefix*, 通道 *points*

Output: 数据点发送到通道

```

1 打开文件 prefix_base.fvecs
2 count  $\leftarrow$  1
3 while 未到文件结尾 do
4     读取维度 d, 若  $d \neq 128$ , 抛出异常
5     读取 128 维向量 vec
6     向 points 发送 job{vector: vec, id: count}
7     count  $\leftarrow$  count + 1
8 关闭 points 通道
    
```

loadDataFromFvec 函数用于从磁盘上的 siftbase.fvecs 格式文件中逐条读取高维向量数据, 并将每个数据点封装后 (包含向量和编号) 依次发送到并发通道, 以便后续处理。

3.6.2 加载查询点和真实邻居

loadQueriesFromFvec 从磁盘批量加载检索实验的查询点及其对应的真实近邻, 为后续索引算法的召回率评价提供基础数据支撑。

3.7 性能优化

3.7.1 SIMD 加速

HNSW 算法在构建和搜索时, 需要大量计算欧氏距离 (L2 距离), 而这部分计算通常是性能瓶颈。为此实现了基于 AVX 指令集的汇编加速。AVX 指令集是现代 CPU 支持的 SIMD 指令集, 可以用于加速数据的并行处理, 特别适合大批量的数值计算任务, 极大提升了距离计算的速度。AVX 指令集一次可以处理 8 个甚至 16 个 float32 元素, 这样可以大幅减少循环次数和指令数量。由于距离计算是 HNSW 算法的核心操作, SIMD 加速让整个索引的构建和查询速度提升数倍甚至数量级, 能够支撑百万级甚至更大规模

算法 3-12: loadQueriesFromFvec 伪代码

Input: 文件前缀 *prefix*

Output: 查询点 *queries*, 真实邻居 *truth*

```

1 打开 prefix_query.fvecs 文件
2 while 未到文件结尾 do
3   读取维度  $d$ , 若  $d \neq 128$  抛出异常
4   读取 128 维向量, 加入 queries
5 打开 prefix_groundtruth.ivecs 文件
6 while 未到文件结尾 do
7   读取维度  $d$ , 若  $d \neq 100$  抛出异常
8   读取 100 个邻居编号, 加入 truth
9 if 查询点数与真实邻居数不一致 then
10  抛出异常
11 return queries, truth

```

的向量检索任务, 这种方式相比传统的逐元素循环, 能极大提升高维向量距离计算的速度, 是该向量搜索系统的关键优化手段。

具体流程如下: 每次循环用 `vmovaps` 指令将 8 个 `float32` 元素从内存加载到 YMM 寄存器; 用 `vsubps` 并行计算两个向量的差值; 用 `vmulps` 并行计算差值的平方; 用 `vaddps` 并行累加结果; 通过循环展开和寄存器操作, 批量处理更多数据, 减少循环次数和内存访问; 最后将所有寄存器中的结果归约 (横向相加) 得到最终的距离和。

3.7.2 并发

`search` 用于并行地对多个查询点执行近邻搜索, 并统计搜索结果的准确率, 通过对比搜索结果和真实近邻集合, 计算整体的 10-NN 精度。在检索评测时, `search` 函数根据 CPU 核心数将查询任务分块, 分配给多个 goroutine 并行处理。每个 goroutine 负责一部分查询点的搜索和精度统计, 最终通过原子操作 `atomic.AddInt32` 安全地累加正确结果数。这样可以充分利用多核资源, 并大幅提升批量查询的吞吐量, 这样的设计充分发挥了多核 CPU 的计算能力, 使得无论是大规模索引构建还是高并发查询, 都能极大地提升响应速

算法 3-13: search 伪代码

Input: 索引 h , 查询集 $queries$, 真实近邻 $truth$, 搜索宽度 $efSearch$

Output: 10-NN 精度

```

1 初始化正确命中计数  $p$ 
2 将  $queries, truth$  平均分配给每个 CPU
3 为每个 CPU 启动一个线程:
4   for 每个分配的查询  $j$  do
度。5   |    $results \leftarrow h.Search(queries[j], efSearch, 10)$ 
6   |   保留  $results$  前 10 个结果
7   |   for 每个  $item$  in  $results$  do
8   |   |   if  $item.ID - 1$  在  $truth[j]$  then
9   |   |   |    $p \leftarrow p + 1$ 
10 等待所有线程完成
11 return  $p / (10 \times \text{查询数})$ 
```

算法 3-14: buildFromChan 伪代码

Input: 索引 h , 数据点通道 $points$

Output: 并发构建索引

```

1 为每个 CPU 启动一个工作线程 goroutine
2 foreach 工作线程 do
3   while 通道未关闭 do
4   |   从  $points$  读取数据点  $job$ 
5   |   调用  $h.Add(job.vector, job.id)$  插入数据
6 等待所有线程完成
```

在索引构建阶段, buildFromChan 采用了“生产者-消费者”模式: 数据加载函数 (如 loadDataFromFvec) 通过 channel 持续发送数据点, 多个 goroutine 并发从 channel 中读取数据并调用 $h.Add$ 方法插入索引。通过 $runtime.NumCPU()$ 动态确定并发协程数, 最大化利用所有 CPU 核心。sync.WaitGroup 用于等待所有数据点插入完成, 保证构建过程的同步和正确性, 不仅提升了处理速度, 还充分发挥了现代多核硬件的性能优势。

3.7.3 位集合池

bitset 是一个开源的 Go 语言第三方库，专门用于高效地管理和操作位集合 (bitset)，支持常见的位运算和集合操作。在本项目的 HNSW 检索系统中，位集合用于记录每次搜索过程中哪些节点已经被访问。由于每次查询都需要一个新的访问标记集合，如果直接频繁创建和销毁 bitset，不仅会带来较大的内存分配和回收开销，还会影响高并发场景下的整体性能，因此定义了位集合池，每个位集合包含一个 busy 标识，通过 get() 方法和 free() 方法实现对位集合池的管理。

这种池化机制能够减少 bitset 的动态分配次数，提升内存利用率和系统吞吐量。特别是在大规模并发检索场景下，BitsetPool 能显著提升整体性能，是本项目高效实现 HNSW 算法不可或缺的优化组件，采用位集合池后，能有效改善系统在高并发和大数据量下的响应速度和资源消耗都有明显改善。

算法 3-15: BitsetPool Get 方法伪代码

Input: 无

Output: 可用的 BitSet 和其索引 id

```

1  加锁
2  for 池中的每个元素  $i$  do
3      if  $\neg pool[i].busy$  then
4          pool[i].busy  $\leftarrow$  true
5          pool[i].b.ClearAll()
6          解锁
7          return  $i, pool[i].b$ 
8   $id \leftarrow$  当前池大小
9  向池中追加空的 poolItem
10 解锁
11 return  $id, pool[id].b$ 

```

第四章 实验结果与分析

4.1 数据集

SIFT 数据集是从图像中提取的高维特征向量集合，经过特定的处理后用于高维空间搜索性能的评估，最初应用于计算机视觉任务，如图像匹配、目标检测等。因其高维且稀疏的特性，使其成为评估高维索引结构的理想数据集。本论文使用了 SIFT 数据集中的 SIFT 的 SIFT1M 数据集，包含 100 万条 128 维的 SIFT 特征向量，每条数据点均为归一化的 float32 数组，此外，数据集还包含 1 万条查询向量 (query)，以及每个查询点对应的 10000 个真实最近邻 (groundtruth)，用于检验检索算法的准确率。所有数据均以 fvecs/ivecs 格式存储，便于高效加载和批量处理。通过加载这些数据，可以全面测试 HNSW 算法在大规模高维检索场景下的构建速度、查询效率和召回率等核心性能指标。

4.2 参数敏感性测试

4.2.1 efconstruction

实验参数设置为 $M=16$ ， $M_0=32$ ， $levelmult=16$ ，分别测试在不同 efsearch 和 efconstruction 下的召回率和搜索时间。

图 4.1 表明随着搜索宽度的增加，搜索时间也在缓慢增加，且速度趋于平缓，同时较高的 efconstruction 也会导致搜索时间的提升 efconstruction 决定在构建阶段的搜索宽度，即候选集和结果集的大小，更大的 efconstruction 让每个节点的连接数变多，图的密集程度增加，查询时需要处理的邻居节点也会更多，缓存命中率降低，也会让查询变慢。

图 4.2 表明搜索准确率随着 efsearch 的增加而增加，准确率在 efsearch 处于个位数时明显低于期待值，说明此时搜索范围较小，难以找到足够多的真实近邻；efsearch 高于个位数后准确率基本高于 0.9，说明算法在搜索过程中能够访问更多的节点，找到更接近真实的最近邻；且较高的 efconstruction 会使准确率明显接近 1，这说明较大的 efconstruction 会让索引结构更加稠密和合理，提升 HNSW 检索的准确性。

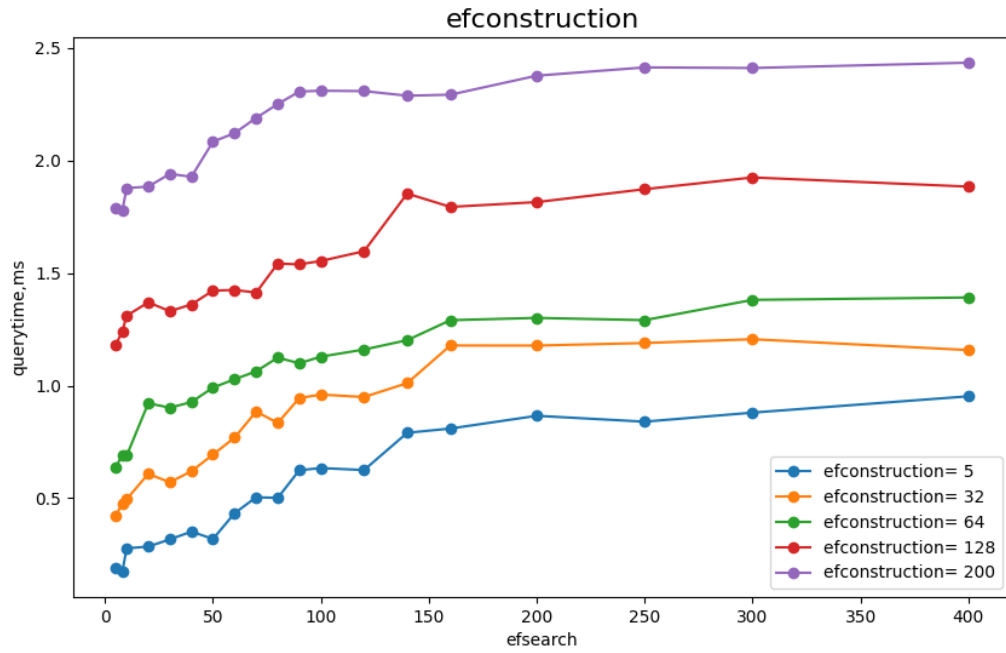


图 4.1 搜索时间与 efconstruction 和 efsearch 的关系

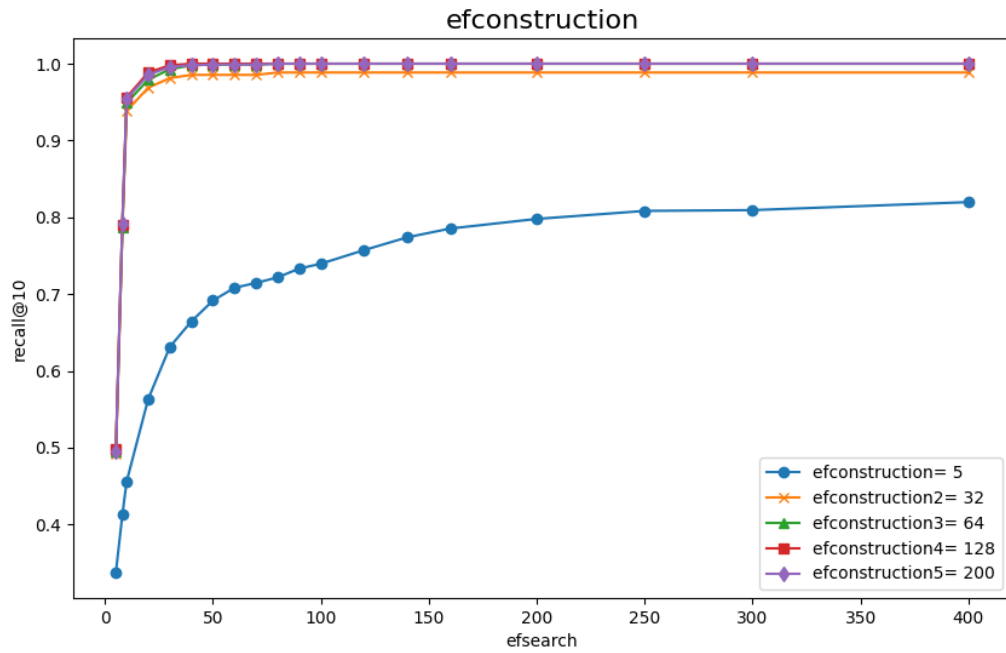


图 4.2 准确率与 efconstruction 和 efsearch 的关系

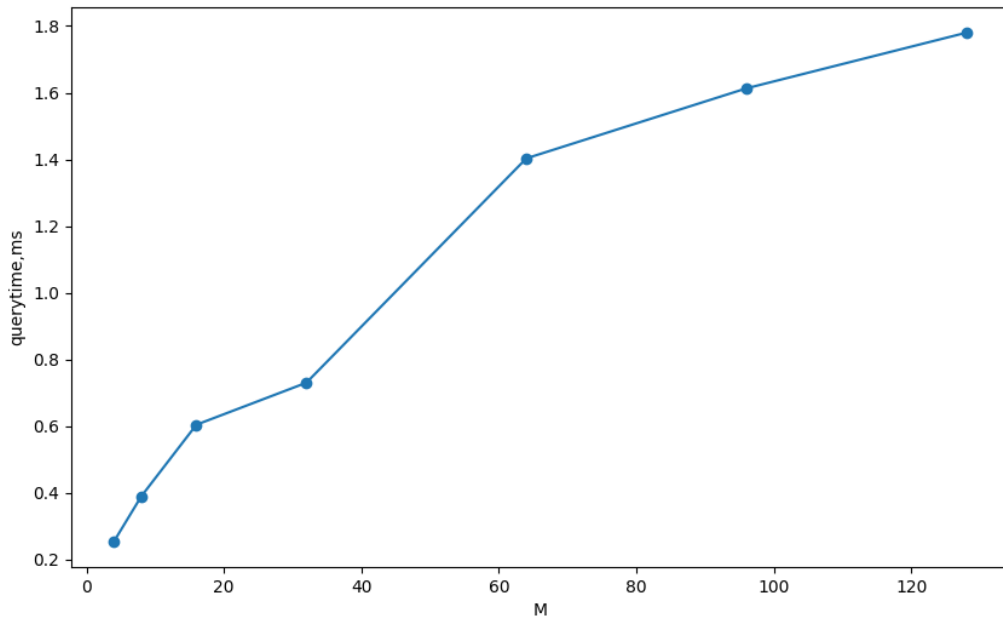


图 4.3 查询时间与 M 的关系

4.2.2 M

我们将最底层最大邻居数设置为 M 的 2 倍，efconstruction 为 60，levelmult 为 10，测试 M 对系统的准确率和查询时间的影响如图 4.3 和图 4.4 所示，查询时间与准确率都随在 M 得增加而增加，M 代表节点最大邻居数，表明 HNSW 索引中每个节点的邻居数随在 M 得增加增多，图结构变得更加稠密。这一方面提升了搜索时遍历到真实近邻的概率，从而使得准确率显著提升；另一方面，也导致每次查询需要访问和比较的节点数量增加，查询时间随之上升。

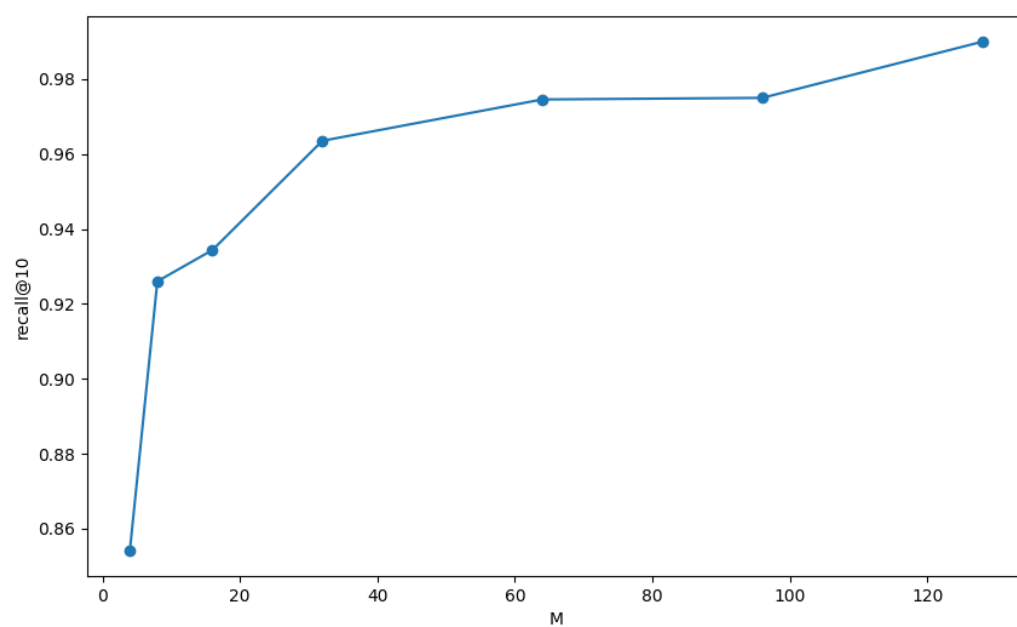


图 4.4 准确率与 M 的关系

第五章 总结与展望

5.1 总结

5.2 展望